Week 5

Aim

The aim of this lab was to understand more about web vulnerabilities through scanning web applications using automated tools. Through scanning these websites, we are then able to identify vulnerabilities and fix the security flaws.

Glossary

Wapiti - Open-source web vulnerability scanner that performs black box testing.

Cross site scripting – A web vulnerability where an attacker injects malicious JavaScript into a website causing the script to run in the browser

SQL injections - Attacks manipulate inputs fields to inject malicious SQL queries into a database.

Command injections – An exploit when untrusted user input is executed as a system level command on a server.

Insecure file upload – Flaw that allows users to upload files without proper validation.

File inclusion - Vulnerability where an application improperly handles file paths allowing for attacks to load local files and therefore exposing sensitive files.

Challenge Explained

For the challenge, in one of my modules known as dynamic web application, we were tasked with creating a fully functioning website with a working link being hosted on goldsmiths' servers. I than thoughts of the idea to test wapiti out on my own website which I created which has login, sanitisation to test my websites vulnerability. However, due to resource reasoning, this website will most likely be shutting down by Thursday to make room for my new coursework in the module, however I have proof that this was a fully functioning website during the time of this test.

Errors and Solutions

One key issue I ran into during this lab was understanding wapiti, specifically, installation and usage. I was very confused when I was introduced to this tool because I did not understand how it operated. Alongside with this I was getting errors which indicated a clash on my virtual environment with my wapiti installation versus my other libraries and modules. How I initially solved this problem was that I created a new folder and created a file with its own independent virtual environment so that I could solely focus on completing the workshop. Later, I went back to the same error only to found out that I inputted ''pip install wapiti'' rather than ''pip install which caused errors because the old wapiti was not supported by python 3 which therefore caused pip to abort.

Another issue I came across was understanding the wapiti report, there was a learning curve since the report was so compact however after taking the time to go through the report and document my findings, I found that the data was laid out and organised well.

Key Takeaways

Website attacks are the most common – Modern web applications often face the constant threat of malicious uploads of injections. Therefore, with websites architecture growing every day, understanding how these attacks operate has been insightful.

Proper input sanitisation and validation is CRITICAL! – Most of these attacks are a result of a common cause which is poor validation of user data. Therefore, implementing strong validation and defensive coding practices could significantly reduce the risk.

Wapiti is great - Working with wapiti showed me how quickly a simple scanner can uncover weaknesses that may not have been obvious through manual inspection. Not just this but learning how wapiti works has also been important as through exploring a career in cybersecurity, it will be critical expose these vulnerabilities from a developer's side before they are exploited by an attacker.